

IF330 – Pemrograman Web

08 - Intro to Web Development with Laravel Framework

Fenina Adline Twince Tobing

I Putu Ananta Yogiswara

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA
SEMESTER GENAP TAHUN AJARAN 2022/2023

REVIEW



- PHP Exception
- Password Hash & Verify
- Random Bytes & bin2hex for Token Generation
- Authentication System

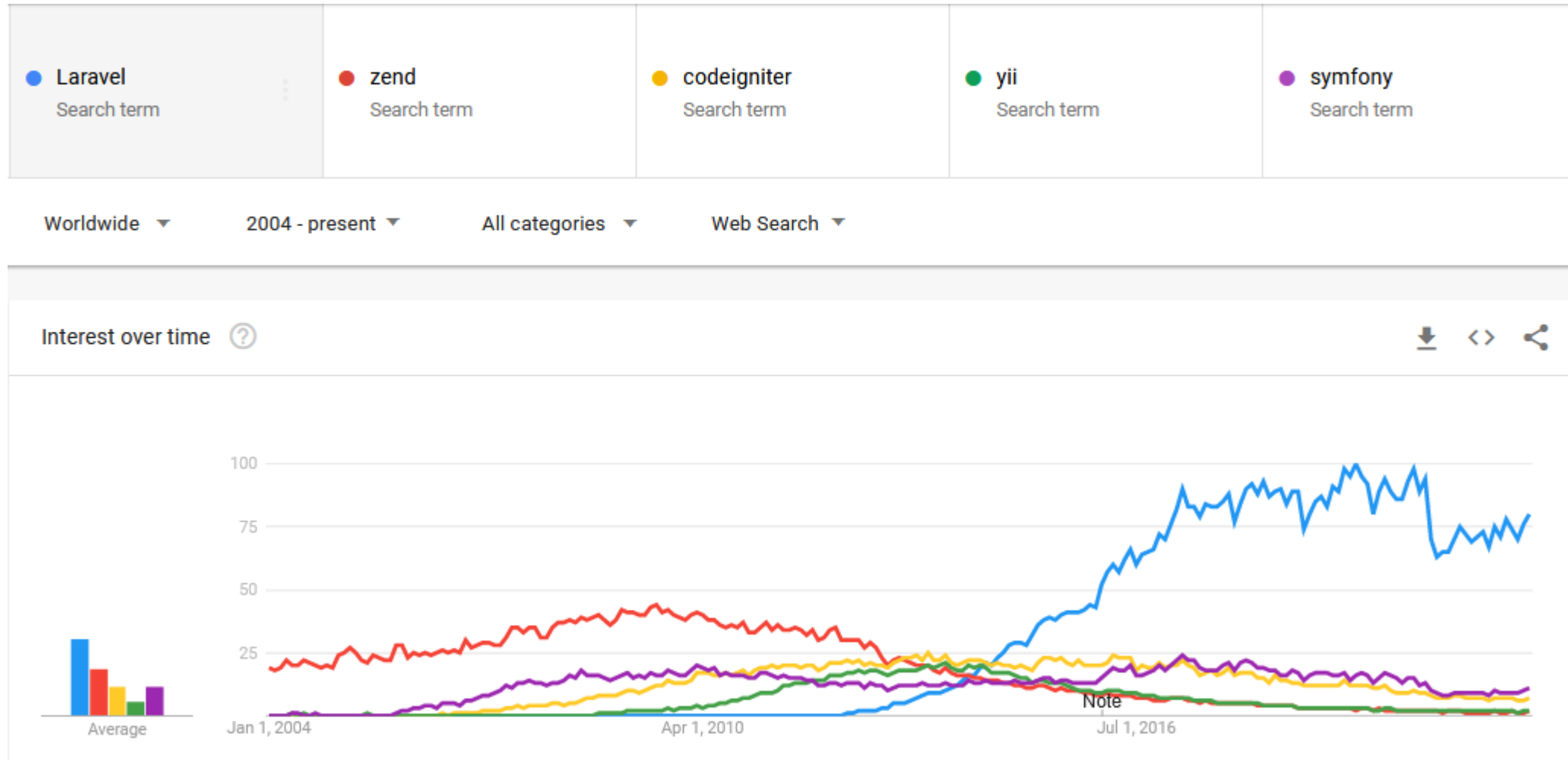
TODAY's OUTLINE

- Introduction to Laravel Framework
- Laravel Installation
- Laravel Project Structure
- Laravel Routing

What is Laravel?

- Laravel is MVC PHP framework created by Taylor Otwell in 2011
- Free open-source license with many contributors worldwide
- One of the best frameworks together with Symfony, CodeIgniter, Yii
- Has powerful features, saving us time
- Uses Symfony packages
- Lets see some statistics

Laravel on Google Trends Worldwide



Features

- **Eloquent ORM** (object-relational mapping) - implements Active Records
- **Query Builder** - helps you to build secured SQL queries
- **Restful controllers** - provides a way for separating the different HTTP requests (GET, POST, DELETE, ETC)
- **Blade template engine** - combines templates with a data model to produce views
- **Migrations** - version control system for database, update your database easier
- **Database seeding** - provides a way to populate database tables with test data used for testing
- **Pagination** - easy to use advanced pagination functionalities
- **Forms security** - provides CSRF token middleware, protecting all the forms

Let's Install Laravel

- Laravel uses Composer to manage its dependencies
- Composer is dependency management tool for PHP, like a library full of books
- NOT like Yum or apt
- Per project tool (vendor folder), not per system
- Install by using the command:

```
composer create-project laravel/laravel my-app
```

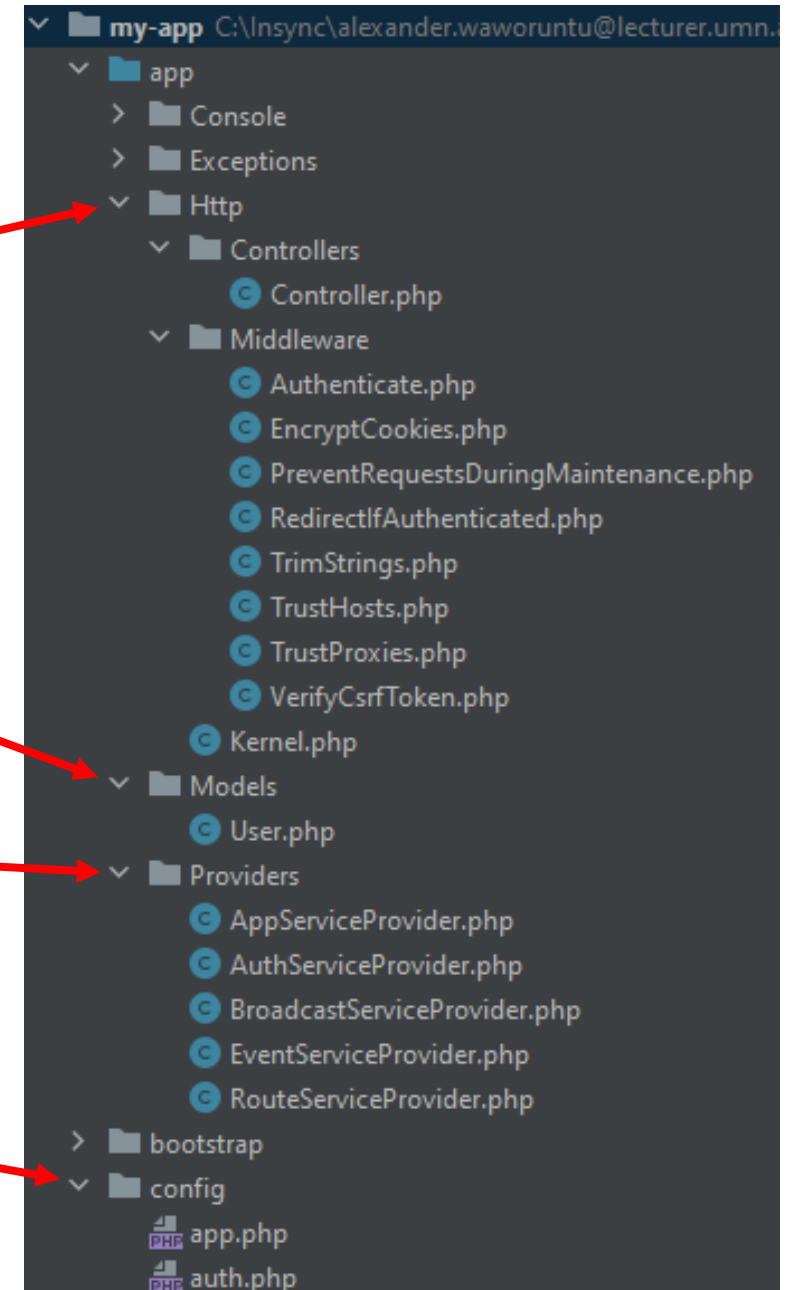
The Structure(1)

app/Http folder contains the **Controllers**, **Middlewares** and **Kernel** files

All the models should be located in **app/Models** folder

The service providers that are bootstrapping functions in our app are located in **app/Providers** folder

All the config files are located in **app/config** folder



The Structure(2)

database folder contains the migrations and seeds

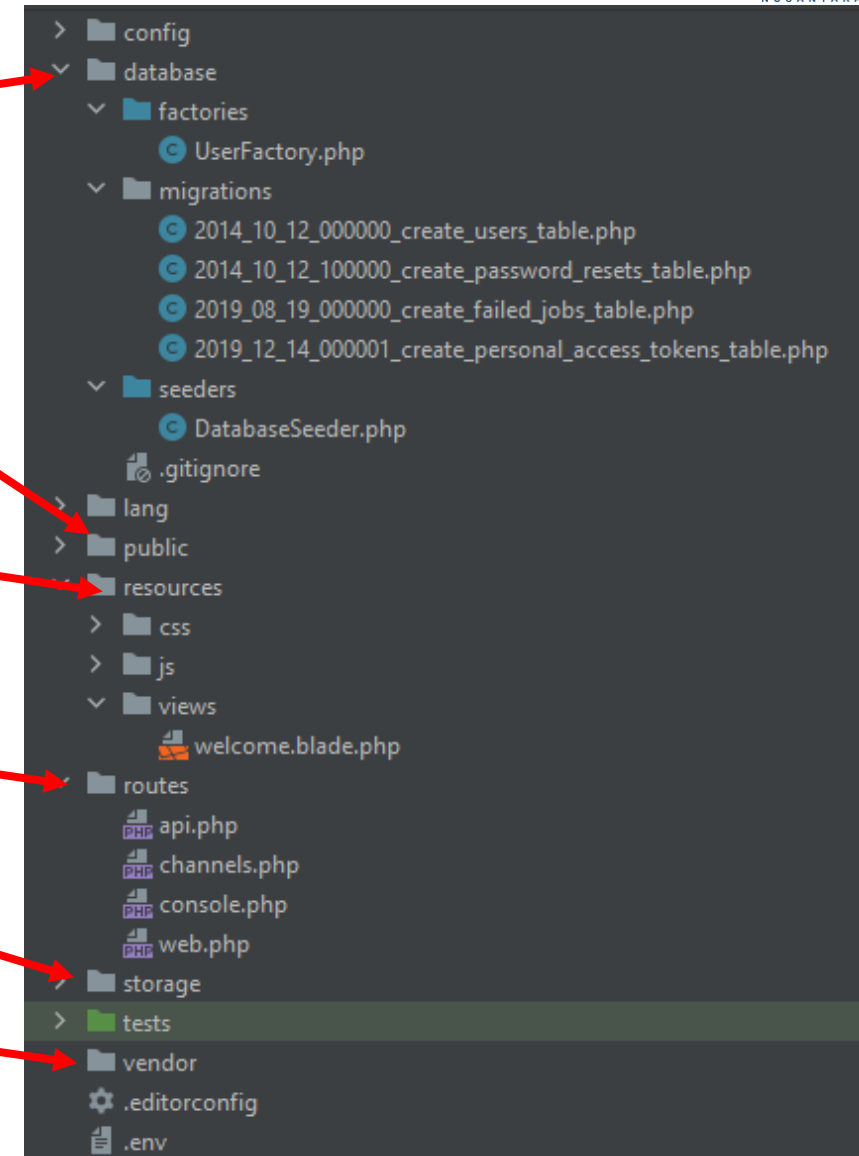
The **public** folder is the actual folder you are opening on the web server.
All JS/CSS/Images/Uploads are located there.

The **resources** folder contains all the translations, views and assets (SASS, LESS, JS) that are compiled into public folder

The **routes** folder contains all the routes for the project

All the logs/cache files are located in **storage** folder

The **vendor** folder contains all the composer packages (dependencies)

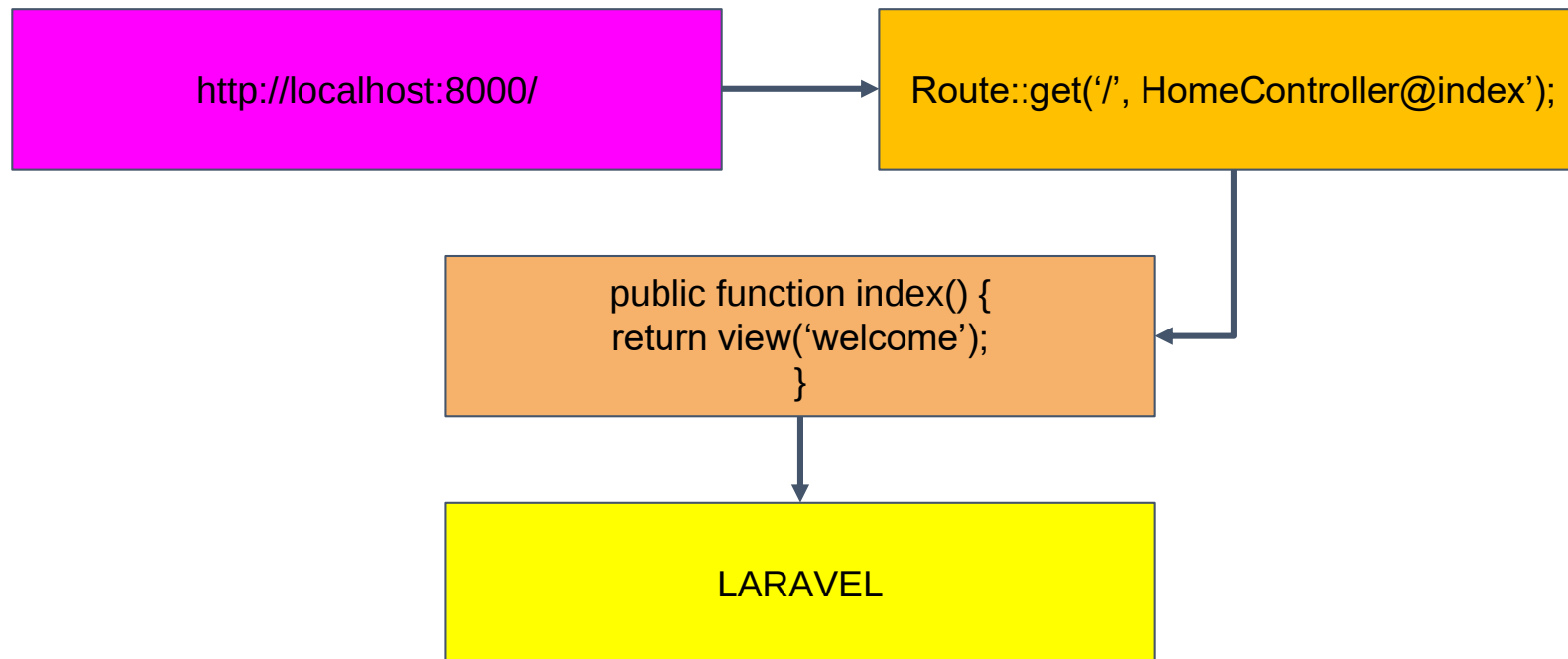


Artisan!

- Artisan is command-line interface for Laravel
- Commands that are saving time
- Generating files with artisan is recommended
- Run `php artisan list` in the console

Routing

- Routing per middleware/prefix or namespace
- Routing per request method (GET, POST, DELETE, etc)
- Be careful with the routing order
- Let's see routing examples



Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

Basic Routing

The most basic Laravel routes accept a URI and a closure, providing a very simple and expressive method of defining routes and behavior without complicated routing configuration files:

```
use Illuminate\Support\Facades\Route;

Route::get('/greeting', function () {
    return 'Hello World';
});
```

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

Redirect Routes

If you are defining a route that redirects to another URI, you may use the `Route::redirect` method. This method provides a convenient shortcut so that you do not have to define a full route or controller for performing a simple redirect:

```
Route::redirect('/here', '/there');
```

By default, `Route::redirect` returns a `302` status code. You may customize the status code using the optional third parameter:

```
Route::redirect('/here', '/there', 301);
```

Or, you may use the `Route::permanentRedirect` method to return a `301` status code:

```
Route::permanentRedirect('/here', '/there');
```

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

View Routes

If your route only needs to return a [view](#), you may use the `Route::view` method. Like the `redirect` method, this method provides a simple shortcut so that you do not have to define a full route or controller. The `view` method accepts a URI as its first argument and a view name as its second argument. In addition, you may provide an array of data to pass to the view as an optional third argument:

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => 'Taylor']);
```

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

The Route List

The `route:list` Artisan command can easily provide an overview of all of the routes that are defined by your application:

```
php artisan route:list
```

By default, the route middleware that are assigned to each route will not be displayed in the `route:list` output; however, you can instruct Laravel to display the route middleware by adding the `-v` option to the command:

```
php artisan route:list -v
```

In addition, you may instruct Laravel to hide any routes that are defined by third-party packages by providing the `--except-vendor` option when executing the `route:list` command:

```
php artisan route:list --except-vendor
```

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

Route Parameters

Required Parameters

Sometimes you will need to capture segments of the URI within your route. For example, you may need to capture a user's ID from the URL. You may do so by defining route parameters:

```
Route::get('/user/{id}', function ($id) {  
    return 'User '.$id;  
});
```

You may define as many route parameters as required by your route:

```
Route::get('/posts/{post}/comments/{comment}', function ($postId, $commentId) {  
    //  
});
```

Route parameters are always encased within `{ }` braces and should consist of alphabetic characters. Underscores (`_`) are also acceptable within route parameter names. Route parameters are injected into route callbacks / controllers based on their order - the names of the route callback / controller arguments do not matter.

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

Optional Parameters

Occasionally you may need to specify a route parameter that may not always be present in the URI. You may do so by placing a `?` mark after the parameter name. Make sure to give the route's corresponding variable a default value:

```
Route::get('/user/{name?}', function ($name = null) {  
    return $name;  
});  
  
Route::get('/user/{name?}', function ($name = 'John') {  
    return $name;  
});
```

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

Regular Expression Constraints

You may constrain the format of your route parameters using the `where` method on a route instance. The `where` method accepts the name of the parameter and a regular expression defining how the parameter should be constrained:

```
Route::get('/user/{name}', function ($name) {
    //
})->where('name', '[A-Za-z]+');

Route::get('/user/{id}', function ($id) {
    //
})->where('id', '[0-9]+');

Route::get('/user/{id}/{name}', function ($id, $name) {
    //
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

For convenience, some commonly used regular expression patterns have helper methods that allow you to quickly add pattern constraints to your routes:

```
Route::get('/user/{id}/{name}', function ($id, $name) {
    //
})->whereNumber('id')->whereAlpha('name');

Route::get('/user/{name}', function ($name) {
    //
})->whereAlphaNumeric('name');

Route::get('/user/{id}', function ($id) {
    //
})->whereUuid('id');
```

If the incoming request does not match the route pattern constraints, a 404 HTTP response will be returned.

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

Encoded Forward Slashes

The Laravel routing component allows all characters except `/` to be present within route parameter values. You must explicitly allow `/` to be part of your placeholder using a `where` condition regular expression:

```
Route::get('/search/{search}', function ($search) {  
    return $search;  
})->where('search', '.*');
```



Encoded forward slashes are only supported within the last route segment.

Routing Documentation

<https://laravel.com/docs/9.x/routing#main-content>

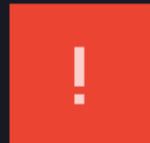
Named Routes

Named routes allow the convenient generation of URLs or redirects for specific routes. You may specify a name for a route by chaining the `name` method onto the route definition:

```
Route::get('/user/profile', function () {  
    //  
})->name('profile');
```

You may also specify route names for controller actions:

```
Route::get(  
    '/user/profile',  
    [UserProfileController::class, 'show']  
)->name('profile');
```



Route names should always be unique.

SUMMARY

After finishing this module, you should be able:

- to understand what is Laravel Framework
- to understand how to install Laravel Framework
- to understand the structure of Laravel Project
- to understand how to implement Routing in Laravel

NEXT WEEK'S OUTLINE



- Laravel Controller
- Laravel View

REFERENCES

<https://laravel.com/docs/9.x>

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.



Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.